

Chapter 4

C Program Control

C How to Program, 9/e, GE

Objectives

In this chapter, you'll learn:

- The essentials of counter-controlled iteration.
- To use the `for` and `do...while` iteration statements to execute statements repeatedly.
- To understand multiple selection using the `switch` selection statement.
- To use the `break` and `continue` statements to alter the flow of control.
- To use the logical operators to form complex conditional expressions in control statements.
- To avoid the consequences of confusing the equality and assignment operators.

4.1 Introduction

- In this chapter
 - iteration is considered in greater detail
 - additional iteration control statements, namely the `for` and the `do...while`
 - `switch` multiple-selection statement
 - `break` statement for exiting immediately from certain control statements
 - `continue` statement for skipping the remainder of the body of an iteration statement and proceeding with the next iteration of the loop.
 - `Logical operators` used for combining conditions
 - Summary of the principles of structured programming as presented in Chapter 3 and 4.

4.2 Iteration Essentials

- A loop is a group of instructions the computer executes repeatedly while some **loop-continuation condition** remains **true**.
- We've discussed two means of iteration:
 - **Counter-controlled** iteration
 - Counter-controlled iteration is sometimes called **definite iteration** because we **know in advance** exactly how many times the loop will be executed.
 - **Sentinel-controlled** iteration
 - Sentinel-controlled iteration is sometimes called **indefinite iteration** because it's **not known in advance** how many times the loop will be executed.

4.2 Iteration Essentials (Cont.)

- In counter-controlled iteration, a **control variable** is used to count the number of iterations.
 - The control variable is **incremented** (usually by 1) each time the group of instructions is performed.
 - When the value of the control variable indicates that the **correct number** of iterations has been performed, the loop **terminates** and **execution** continues with the statement after the iteration statement.

4.2 Iteration Essentials (Cont.)

- **Sentinel values** are used to control iteration when:
 - The precise number of iterations **isn't known in advance**, and
 - The loop includes statements that obtain data each time the loop is performed.
- The **sentinel** value indicates “**end of data**.”
- The **sentinel** is entered after all regular data items have been supplied to the program.
- **Sentinels** must be distinct from regular data items.

Review Exercises

► Review Exercises

- Fill in the blanks in each of the following statements.

a) Counter-controlled repetition is also known as _____ repetition because it is known in advance how many times the loop will be executed.

ANS:

b) Sentinel-controlled repetition is also known as _____ repetition because it is not known in advance how many times the loop will be executed.

ANS:

Review Exercises

- Fill in the blanks in each of the following statements.

a) Counter-controlled repetition is also known as _____ repetition because it is known in advance how many times the loop will be executed.

ANS: definite.

b) Sentinel-controlled repetition is also known as _____ repetition because it is not known in advance how many times the loop will be executed.

ANS: indefinite.

4.3 Counter-Controlled Iteration

- Counter-controlled iteration requires:
 - The **name** of a control variable (or loop **counter**).
 - The **initial value** of the control variable.
 - The **increment** (or **decrement**) by which the control variable is modified each time through the loop.
 - The condition that tests for the **final value** of the control variable (i.e., whether looping should continue).

4.3 Counter-Controlled Iteration (Cont.)

- Prints the numbers from 1 to 10.

```
1 // Fig. 4.1: fig04_01.c
2 // Counter-controlled iteration.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     unsigned int counter = 1; // initialization
8
9     while (counter <= 10) { // iteration condition
10         printf ("%u\n", counter);
11         ++counter; // increment
12     }
13 }
```

```
1
2
3
4
5
6
7
8
9
10
```

- The definition
 - unsigned int counter = 1; // initialization**
 - names the control variable (counter)
 - defines it to be an integer
 - reserves memory space for it
 - sets it to an initial value of 1.
- This definition is not an executable statement.

Program Demo

- Fig. 4.1

Review Exercises

- Fill in the blanks in each of the following statements.
In counter-controlled repetition, a(n) _____ is used to count the number of times a group of instructions should be repeated.

ANS:

Review Exercises

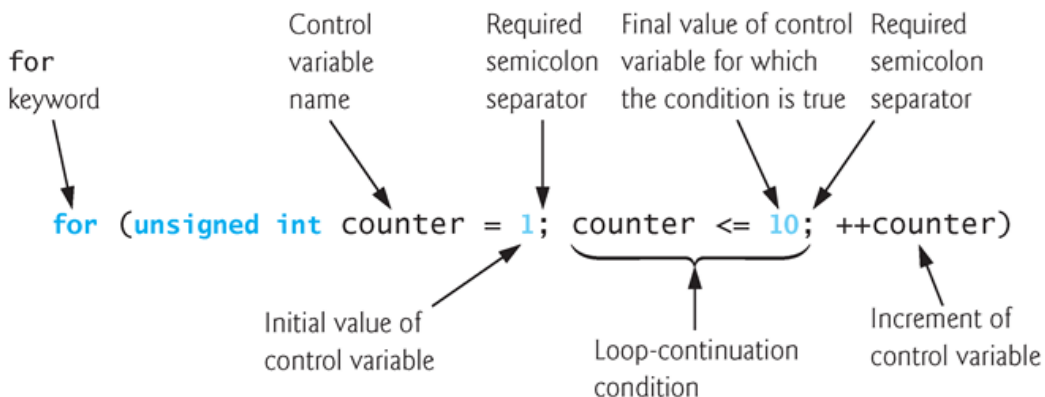
- Fill in the blanks in each of the following statements.
In counter-controlled repetition, a(n) _____ is used to count the number of times a group of instructions should be repeated.
ANS: control variable or counter.

4.4 for Iteration Statement

- The for iteration statement handles all the details of counter-controlled iteration.

```
1 // Fig. 4.1: fig04_01.c
2 // Counter-controlled iteration.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     unsigned int counter = 1; // initialization
8
9     while (counter <= 10) { // iteration condition
10         printf ("%u\n", counter);
11         ++counter; // increment
12     }
13 }
```

```
1 // Fig. 4.2: fig04_02.c
2 // Counter-controlled iteration with the for statement.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     // initialization, iteration condition, and increment
8     // are all included in the for statement header.
9     for (unsigned int counter = 1; counter <= 10; ++counter) {
10         printf("%u\n", counter);
11     }
12 }
```



Off-By-One Errors

If you incorrectly wrote `counter < 10`, then the loop would be executed only 9 times.

4.4 for Iteration Statement (Cont.)

General Format of a for Statement

- The **general format** of the for statement is

```
for (initialization; condition; increment) {  
    statement  
}
```

 - The ***initialization*** expression initializes the loop-control variable (and might define it)
 - The ***condition*** expression is the loop-continuation condition
 - The ***increment*** expression increments the control variable.
- **for** statement can be represented with an equivalent **while** statement:

```
initialization;  
while (condition) {  
    statement  
    increment;  
}
```

Program Demo

- Fig. 4.2

4.5 for Statement: Notes and Observations

- The initialization, loop-continuation condition and increment can contain **arithmetic expressions**.
- For example, if $x = 2$ and $y = 10$, the statement

```
for (j = x; j <= 4 * x * y; j += y / x)
```

is equivalent to the statement

```
for (j = 2; j <= 80; j += 5)
```

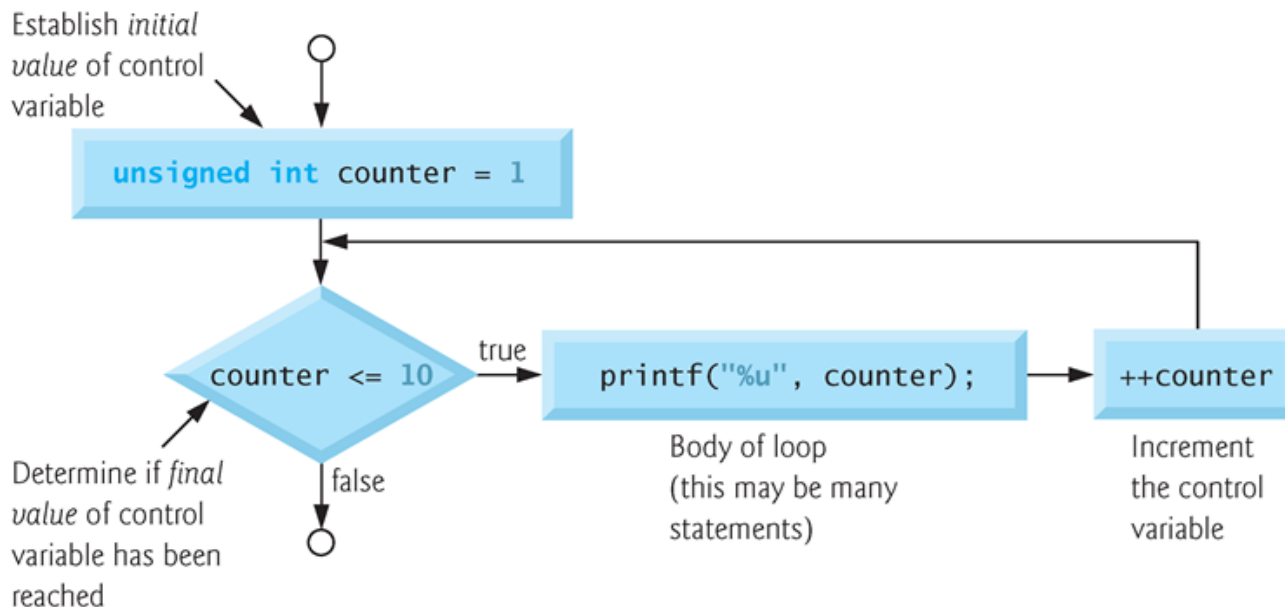
 - The “increment” may be **negative** (in which case it’s really a decrement and the loop actually counts downward).
 - If the loop-continuation condition is **initially false**, the loop body **does not execute**.
 - Instead, execution proceeds with the statement following the **for** statement.

4.5 for Statement: Notes and Observations (cont.)

- The for statement is **flowcharted** much like the while statement.

```
for (counter = 1; counter <= 10; ++counter)  
    printf("%u", counter);
```

- This flowchart makes it clear that the initialization occurs only **once** and that incrementing occurs *after* the body statement is performed.



4.6 Examples Using the for Statement

- The following examples show methods of varying the control variable in a **for** statement.
 - Vary the control variable from 1 to 100 in increments of 1.
for (**i** = **1**; **i** <= **100**; ++ **i**)
 - Vary the control variable from 100 to 1 in increments of -1 (decrements of 1).
for (**i** = **100**; **i** >= **1**; --**i**)
 - Vary the control variable from 7 to 77 in steps of 7.
for (**i** = **7**; **i** <= **77**; **i** += **7**)
 - Vary the control variable from 20 to 2 in steps of -2.
for (**i** = **20**; **i** >= **2**; **i** -= **2**)
 - Vary the control variable over the following sequence of values: 2, 5, 8, 11, 14, 17.
for (**j** = **2**; **j** <= **17**; **j** += **3**)
 - Vary the control variable over the following sequence of values: 44, 33, 22, 11, 0.
for (**j** = **44**; **j** >= **0**; **j** -= **11**)

4.6 Examples Using the for Statement (Cont.)

Application: Summing the Even Integers from 2 to 100

- Figure 4.5 uses the for statement to sum all the **even** integers from 2 to 100.

```
1 // Fig. 4.5: fig04_05.c
2 // Summation with for.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     unsigned int sum = 0; // initialize sum
8
9     for (unsigned int number = 2; number <= 100; number += 2) {
10         sum += number; // add number to sum
11     }
12
13     printf("Sum is %u\n", sum);
14 }
```

Sum is 2550

4.6 Examples Using the for Statement (Cont.)

- The body of the for statement in Fig. 4.5 could actually be merged into the rightmost portion of the for header by using the **comma** operator as follows:

```
for (number = 2; number <= 100; sum += number, number += 2)  
    ; // empty statement
```

- The initialization `sum = 0` could also be merged into the initialization section of the for.

```
1 // Fig. 4.5: fig04_05.c  
2 // Summation with for.  
3 #include <stdio.h>  
4  
5 int main(void)  
6 {  
7     unsigned int sum = 0; // initialize sum  
8  
9     for (unsigned int number = 2; number <= 100; number += 2) {  
10         sum += number; // add number to sum  
11     }  
12  
13     printf("Sum is %u\n", sum);  
14 }
```



```
Sum is 2550
```

4.6 Examples Using the for Statement (Cont.)

Application: Compound-Interest Calculations

- Consider the following problem statement:
 - A person invests \$1000.00 in a **savings** account yielding 5% **interest**.
 - Assuming that all interest is left on deposit in the account, **calculate** and **print** the amount of money in the account at the end of **each year** for 10 years.
 - Use the following formula for determining these amounts:
$$a = p(1 + r)^n$$
where
 - p is the original amount invested (i.e., the principal)
 - r is the annual interest rate
 - n is the number of years
 - a is the amount on deposit at the end of the nth year.
- This problem involves a loop that performs the indicated calculation for each of the 10 years the money remains on deposit.

```

1 // Fig. 4.6: fig04_06.c
2 // Calculating compound interest.
3 #include <stdio.h>
4 #include <math.h>
5
6 int main(void)
7 {
8     double principal = 1000.0; // starting principal
9     double rate = .05; // annual interest rate
10
11     // output table column heads
12     printf("%4s%21s\n", "Year", "Amount on deposit");
13
14     // calculate amount on deposit for each of ten years
15     for (unsigned int year = 1; year <= 10; ++year) {
16
17         // calculate new amount for specified year
18         double amount = principal * pow(1.0 + rate, year);
19
20         // output one table row
21         printf("%4u%21.2f\n", year, amount);
22     }
23 }

```

called pow's function prototype

$a = p(1 + r)^n$
 p is the original amount invested (i.e., the principal)
 r is the annual interest rate
 n is the number of years
 a is the amount on deposit at the end of the n^{th} year.

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

- If the number of characters displayed is less than the **field width**, then the value will automatically be *right justified* in the field.
- To *left justify* a value in a field, place a - (minus sign) between the % and the **field width**.
- To left justify
 - integers (such as in %-6d)
 - character strings (such as in %-8s)

Program Demo

- Fig. 4.6

Review Exercises

Write a statement or a set of statements to accomplish the following tasks:

- ▶ Sum the odd integers between 1 and 99 using a for statement. Assume the integer variables sum and count have been defined.

ANS:

Review Exercises

Write a statement or a set of statements to accomplish the following tasks:

- ▶ Sum the odd integers between 1 and 99 using a for statement. Assume the integer variables sum and count have been defined.

ANS:

```
sum = 0;  
for ( count = 1; count <= 99; count += 2 )  
    sum += count;
```

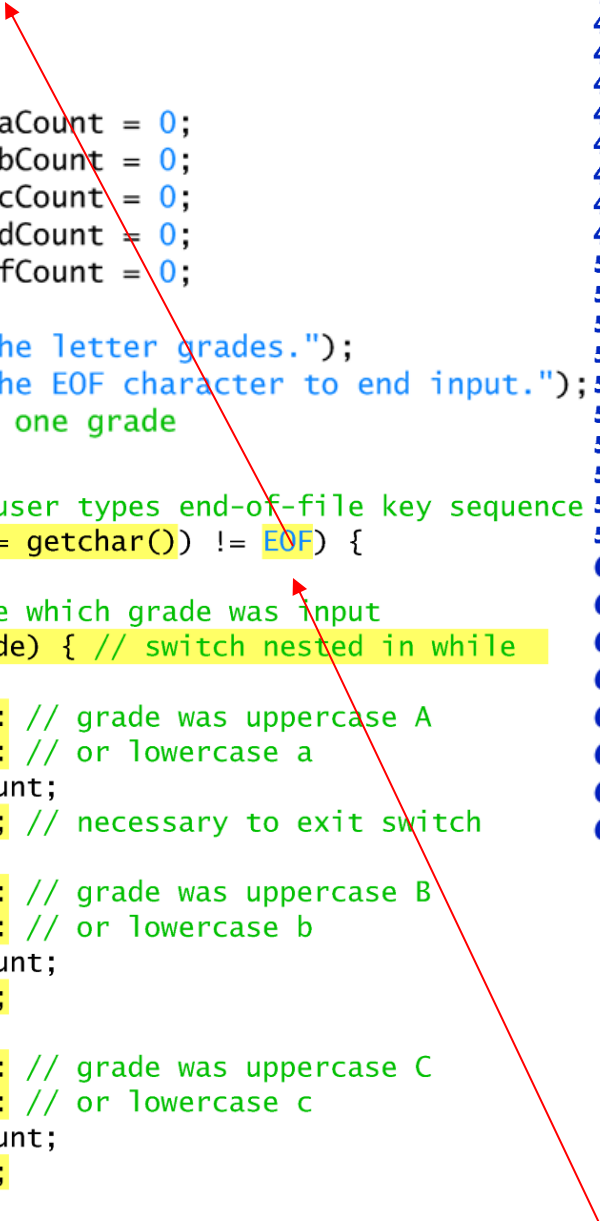
4.7 switch Multiple-Selection Statement

- An algorithm will contain a *series of decisions*
 - A variable or expression is tested separately for each of the constant integral values it may assume, and different actions are taken.
 - This is called *multiple selection*.
- C provides the `switch` multiple-selection statement to handle such decision making.
 - The `switch` statement consists of a *series of case labels*, an *optional default case* and *statements to execute for each case*.
- Figure 4.7 uses `switch` to **count** the number of each different letter grade students earned on an exam.

```

1 // Fig. 4.7: fig04_07.c
2 // Counting letter grades with switch.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     unsigned int aCount = 0;
8     unsigned int bCount = 0;
9     unsigned int cCount = 0;
10    unsigned int dCount = 0;
11    unsigned int fCount = 0;
12
13    puts("Enter the letter grades.");
14    puts("Enter the EOF character to end input.");
15    int grade; // one grade
16
17    // loop until user types end-of-file key sequence
18    while ((grade = getchar()) != EOF) {
19
20        // determine which grade was input
21        switch (grade) { // switch nested in while
22
23            case 'A': // grade was uppercase A
24            case 'a': // or lowercase a
25                ++aCount;
26                break; // necessary to exit switch
27
28            case 'B': // grade was uppercase B
29            case 'b': // or lowercase b
30                ++bCount;
31                break;
32
33            case 'C': // grade was uppercase C
34            case 'c': // or lowercase c
35                ++cCount;
36                break;
37
38            case 'D': // grade was uppercase D
39            case 'd': // or lowercase d
40                ++dCount;
41                break;
42
43            case 'F': // grade was uppercase F
44            case 'f': // or lowercase f
45                ++fCount;
46                break;
47
48            case '\n': // ignore newlines,
49            case '\t': // tabs,
50            case ' ': // and spaces in input
51                break;
52
53            default: // catch all other characters
54                printf("%s", "Incorrect letter grade entered.");
55                puts(" Enter a new grade.");
56                break; // optional; will exit switch anyway
57        }
58    } // end while
59
60    // output summary of results
61    puts("\nTotals for each letter grade are:");
62    printf("A: %u\n", aCount);
63    printf("B: %u\n", bCount);
64    printf("C: %u\n", cCount);
65    printf("D: %u\n", dCount);
66    printf("F: %u\n", fCount);
67 }

```



We use EOF (which normally has the value -1) as the sentinel value.

```

Enter the letter grades.
Enter the EOF character to end input.
a
b
c
C
A
d
f
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
b
^Z ——— Not all systems display a representation of the EOF character

Totals for each letter grade are:
A: 3
B: 2
C: 3
D: 2
F: 1

```

4.7 switch Multiple-Selection Statement (Cont.)

Entering the EOF Indicator

- On Linux/UNIX/Mac OS X systems, the EOF indicator is entered by typing

<Ctrl> d

on a line by itself.

- This notation *<Ctrl> d* means to press the *Enter* key then simultaneously press both *Ctrl* and *d*.
- On other systems, such as Microsoft Windows, the EOF indicator can be entered by typing

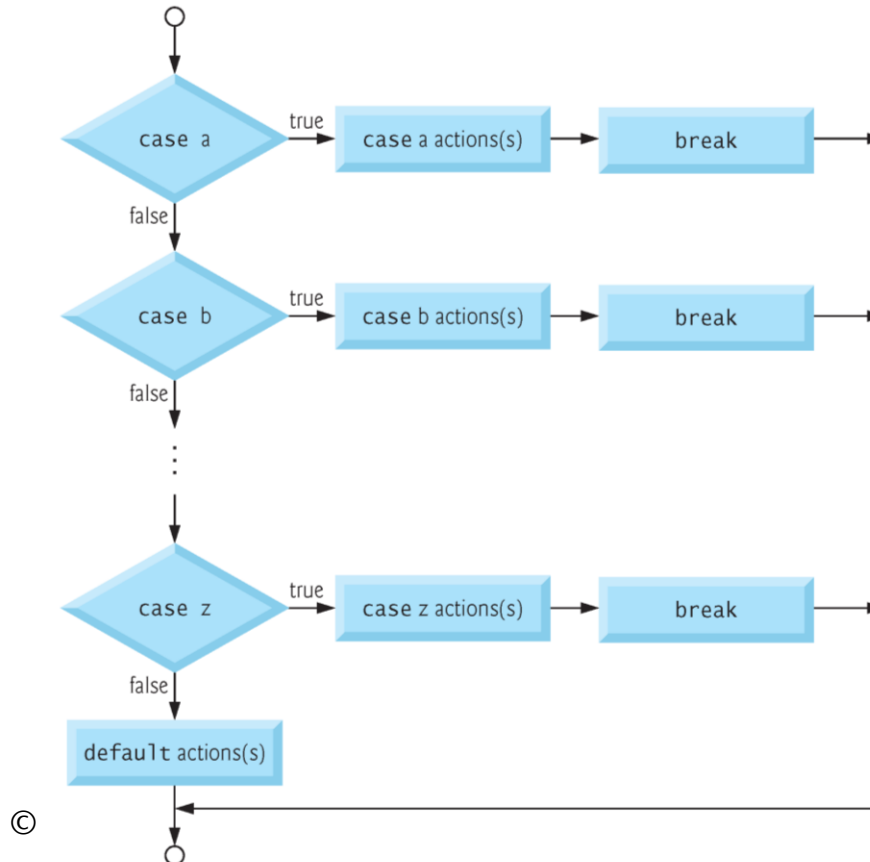
<Ctrl> z

- You may also need to press *Enter* on Windows.

4.7 switch Multiple-Selection Statement (Cont.)

switch Statement Flowchart

- Each case can have one or more actions.
- The switch statement is different from all other control statements in that **braces are not required around multiple actions** in a case of a switch.



4.7 switch Multiple-Selection Statement (Cont.)

- Characters *must* be enclosed within **single quotes** to be recognized as **character constants**
- Characters in **double quotes** are recognized as **strings**.

4.7 switch Multiple-Selection Statement (Cont.)

Notes on Integral Types

- Portable languages like C must have **flexible** data type sizes.
- Different applications may need integers of **different** sizes.
- C provides several data types to represent integers.
- In addition to **int** and **char**
 - C provides types
 - **short int** (which can be abbreviated as **short**)
 - The minimum range of values is -32767 to +32767
 - **long int** (which can be abbreviated as **long**)
 - The minimum range of values is -2147483647 to +2147483647
 - **unsigned**
- The data type **signed char** can be used to represent integers in the range -128 to +127 or any of the characters in the computer's character set.

Program Demo

- Fig. 4.7

4.8 do...while Iteration Statement

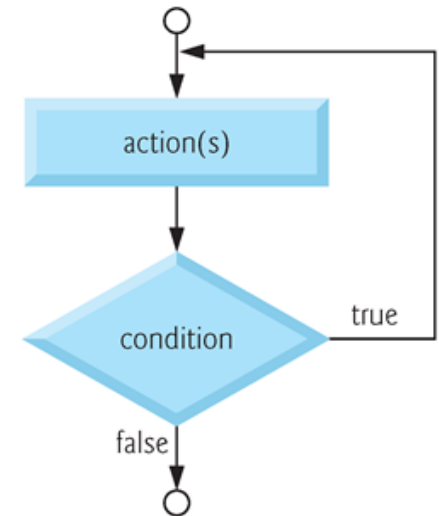
- The do...while iteration statement is similar to the while statement.
 - In the **while** statement, the loop-continuation condition is tested at the **beginning** of the loop before the body of the loop is performed.
 - The **do...while** statement tests the loop-continuation condition *after* the loop body is performed.
 - Therefore, the loop body will be executed **at least once**.
 - When a **do...while** terminates, execution continues with the statement after the **while** clause.

4.8 do...while Iteration Statement (Cont.)

- A do...while with **no** braces around the single-statement body appears as

```
do
    statement
while (condition);
```

```
1  // Fig. 4.9: fig04_09.c
2  // Using the do...while iteration statement.
3  #include <stdio.h>
4
5  int main(void)
6  {
7      unsigned int counter = 1; // initialize counter
8
9      do {
10         printf("%u  ", counter);
11     } while (++counter <= 10);
12 }
```



Program Demo

- Fig. 4.9

4.9 break and continue Statements

- The break and continue statements are used to alter the flow of control.

break Statement

- The break statement, when executed in a while, for, do...while or switch statement, causes an **immediate exit** from that statement.
- Program execution continues with the next statement.
- Common uses of the break statement are to **escape early from a loop** or to **skip the remainder** of a switch statement (as in Fig. 4.7).

```

1 // Fig. 4.7: fig04_07.c
2 // Counting letter grades with switch.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     unsigned int aCount = 0;
8     unsigned int bCount = 0;
9     unsigned int cCount = 0;
10    unsigned int dCount = 0;
11    unsigned int fCount = 0;
12
13    puts("Enter the letter grades.");
14    puts("Enter the EOF character to end input.");
15    int grade; // one grade
16
17    // loop until user types end-of-file key sequence
18    while ((grade = getchar()) != EOF) {
19
20        // determine which grade was input
21        switch (grade) { // switch nested in while
22
23            case 'A': // grade was uppercase A
24            case 'a': // or lowercase a
25                ++aCount;
26                break; // necessary to exit switch
27
28            case 'B': // grade was uppercase B
29            case 'b': // or lowercase b
30                ++bCount;
31                break;
32
33            case 'C': // grade was uppercase C
34            case 'c': // or lowercase c
35                ++cCount;
36                break;
37
38            case 'D': // grade was uppercase D
39            case 'd': // or lowercase d
40                ++dCount;
41                break;
42
43            case 'F': // grade was uppercase F
44            case 'f': // or lowercase f
45                ++fCount;
46                break;
47
48            case '\n': // ignore newlines,
49            case '\t': // tabs,
50            case ' ': // and spaces in input
51                break;
52
53            default: // catch all other characters
54                printf("%s", "Incorrect letter grade entered.");
55                puts(" Enter a new grade.");
56                break; // optional; will exit switch anyway
57        }
58    } // end while
59
60    // output summary of results
61    puts("\nTotals for each letter grade are:");
62    printf("A: %u\n", aCount);
63    printf("B: %u\n", bCount);
64    printf("C: %u\n", cCount);
65    printf("D: %u\n", dCount);
66    printf("F: %u\n", fCount);
67 }

```

```

Enter the letter grades.
Enter the EOF character to end input.
a
b
c
C
A
d
f
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
b
^Z ——— Not all systems display a representation of the EOF character

Totals for each letter grade are:
A: 3
B: 2
C: 3
D: 2
F: 1

```

We use EOF (which normally has the value -1) as the sentinel value.

4.9 break and continue Statements (Cont.)

- Demonstrates the **break** statement in a for iteration statement.

```
1 // Fig. 4.11: fig04_11.c
2 // Using the break statement in a for statement.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     unsigned int x; // declared here so it can be used after loop
8
9     // loop 10 times
10    for (x = 1; x <= 10; ++x) {
11
12        // if x is 5, terminate loop
13        if (x == 5) {
14            break; // break loop only if x is 5
15        }
16
17        printf("%u ", x);
18    }
19
20    printf("\nBroke out of loop at x == %u\n", x);
21 }
```

```
1 2 3 4
Broke out of loop at x == 5
```


4.9 break and continue Statements (Cont.)

- Demonstrates the **continue** statement in a **for** iteration statement.

```
1 // Fig. 4.12: fig04_12.c
2 // Using the continue statement in a for statement.
3 #include <stdio.h>
4
5 int main(void)
6 {
7     // loop 10 times
8     for (unsigned int x = 1; x <= 10; ++x) {
9
10        // if x is 5, continue with next iteration of loop
11        if (x == 5) {
12            continue; // skip remaining code in loop body
13        }
14
15        printf("%u ", x);
16    }
17
18    puts("\nUsed continue to skip printing the value 5");
19 }
```

```
1 2 3 4 6 7 8 9 10
Used continue to skip printing the value 5
```

Review Exercises

- Fill in the blanks in each of the following statements.
- The _____ statement, when executed in a repetition statement, causes the next iteration of the loop to be performed immediately.

ANS:

- The _____ statement, when executed in a repetition statement or a switch, causes an immediate exit from the statement.

ANS:

- The _____ is used to test a particular variable or expression for each of the constant integral values it may assume.

ANS:

Review Exercises

- Fill in the blanks in each of the following statements.
- The _____ statement, when executed in a repetition statement, causes the next iteration of the loop to be performed immediately.

ANS: continue.

- The _____ statement, when executed in a repetition statement or a switch, causes an immediate exit from the statement.

ANS: break.

- The _____ is used to test a particular variable or expression for each of the constant integral values it may assume.

ANS: switch selection statement.

Program Demo

- Fig. 4.11 and 4.12

4.10 Logical Operators

- C provides *logical operators* that may be used to form **more complex conditions** by combining simple conditions.
- The logical operators are
 - && (logical AND)
 - || (logical OR)
 - ! (logical NOT also called logical negation)

4.10 Logical Operators (Cont.)

Logical AND (&&) Operator

- Two conditions are **both true** before we choose a certain path of execution.
- Uses the logical operator && as follows:

```
if (gender == 1 && age >= 65)
    ++seniorFemales;
```

- This if statement contains *two* simple conditions.
- The condition `gender == 1` might be evaluated, for example, to determine if a person is a female.
- The condition `age >= 65` is evaluated to determine whether a person is a senior citizen.
- The two simple conditions are evaluated **first** because the precedences of `==` and `>=` are both *higher than* the precedence of `&&`.

```
if (gender == 1 && age >= 65)
    ++seniorFemales;
```

4.10 Logical Operators (Cont.)

- The `if` statement then considers the combined condition

`gender == 1 && age >= 65`

Which is *true* if and only if **both** of the simple conditions are *true*.

- Finally, if this combined condition is true, then the count of `seniorFemales` is incremented by 1.
- If *either* or *both* of the simple conditions are false, then the program **skips** the incrementing and proceeds to the statement following the `if`.

expression1	expression2	expression1 && expression2
0	0	0
0	nonzero	0
nonzero	0	0
nonzero	nonzero	1

- Such tables are often called **truth tables**.
- C evaluates all expressions that include **relational** operators, **equality** operators, and/or **logical** operators to 0 or 1.*
- Although C sets a true value to 1, it **accepts any nonzero value as true**.⁴⁷

Fig. 4.13 | Truth table for the logical AND (&&) operator.

4.10 Logical Operators (Cont.)

Logical OR (| |) Operator

- To ensure at some point in a program that *either or both* of two conditions are *true* before we choose a certain path of execution.
- Uses the `| |` operator as in the following program segment

```
if (semesterAverage >= 90 || finalExam >= 90)
    printf("Student grade is A");
```

expression1	expression2	expression1 expression2
0	0	0
0	nonzero	1
nonzero	0	1
nonzero	nonzero	1

Fig. 4.14 | Truth table for the logical OR (| |) operator.

4.10 Logical Operators (Cont.)

Logical Negation (!) Operator

- C provides ! (logical negation) to enable you to “reverse” the meaning of a condition.
- The logical negation operator has only a single condition as an operand (and is therefore a unary operator).
- Placed before a condition when we’re interested in choosing a path of execution if the original condition (without the logical negation operator) is false, such as in the following program segment:

```
if (!(grade == sentinelValue))  
    printf("The next grade is %f\n", grade);
```

expression	!expression
0	1
nonzero	0

Fig. 4.15 | Truth table for operator ! (logical negation).

4.10 Logical Operators (Cont.)

Summary of Operator Precedence and Associativity

Operators	Associativity	Type
++ (<i>postfix</i>) -- (<i>postfix</i>)	right to left	postfix
+ - ! ++ (<i>prefix</i>) -- (<i>prefix</i>) (<i>type</i>)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Fig. 4.16 | Operator precedence and associativity.

- The && operator has a **higher precedence** than ||.
- Both operators associate from **left to right**.

4.11 Confusing Equality (==) and Assignment (=) Operators

- operators `==` (equality) and `=` (assignment).
- If the value is `0`, it's treated as `false`, and if the value is `nonzero`, it's treated as `true`.

4.11 Confusing Equality (==) and Assignment (=) Operators (Cont.)

- For example, suppose we intend to write

```
if (payCode == 4)
    printf("%s", "You get a bonus!");
```

but we **accidentally** write

```
if (payCode = 4)
    printf("%s", "You get a bonus!");
```

- The first `if` statement properly awards a bonus to the person whose paycode is **equal** to 4.
- The second `if` statement—the one with the **error**—**evaluates** the **assignment** expression in the `if` condition.

4.11 Confusing Equality (==) and Assignment (=) Operators (Cont.)

lvalues and rvalues

- You'll probably be inclined to write conditions such as **x == 7** with the variable name on the **left** and the constant on the **right**.
- By reversing these terms so that the constant is on the left and the variable name is on the right, as **in 7 == x**, then if you accidentally replace the == operator with =, you'll be **protected** by the compiler.
 - The compiler will treat this as a *syntax error*, because only a variable name can be placed on the left-hand side of an assignment expression.

4.11 Confusing Equality (==) and Assignment (=) Operators (Cont.)

Confusing == and = in Standalone Statements

- The other side of the coin can be equally unpleasant. (~~=~~ → ==)
- Suppose you want to **assign** a value to a variable with a simple statement such as

x = 1;

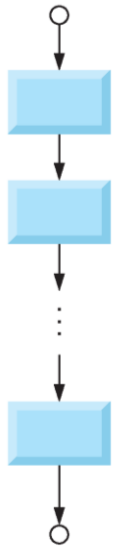
but **instead** write

x == 1;

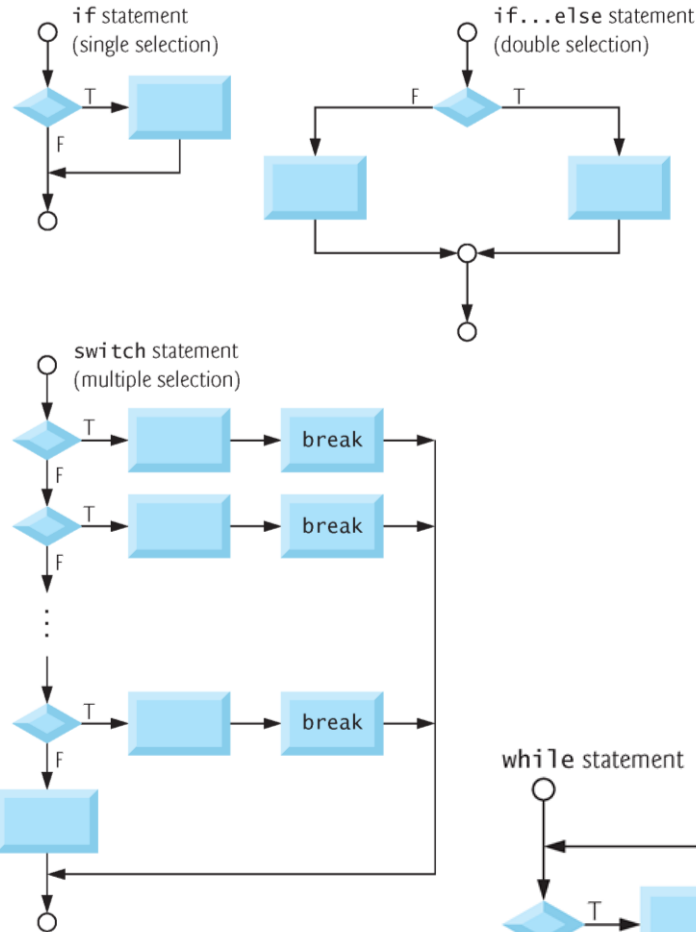
- Here, too, this is **not** a syntax error.
 - If x is **equal** to 1, the condition is **true** and the expression returns the value 1.
 - If x is **not equal** to 1, the condition is **false** and the expression returns the value 0.

4.12 Structured Programming Summary

Sequence

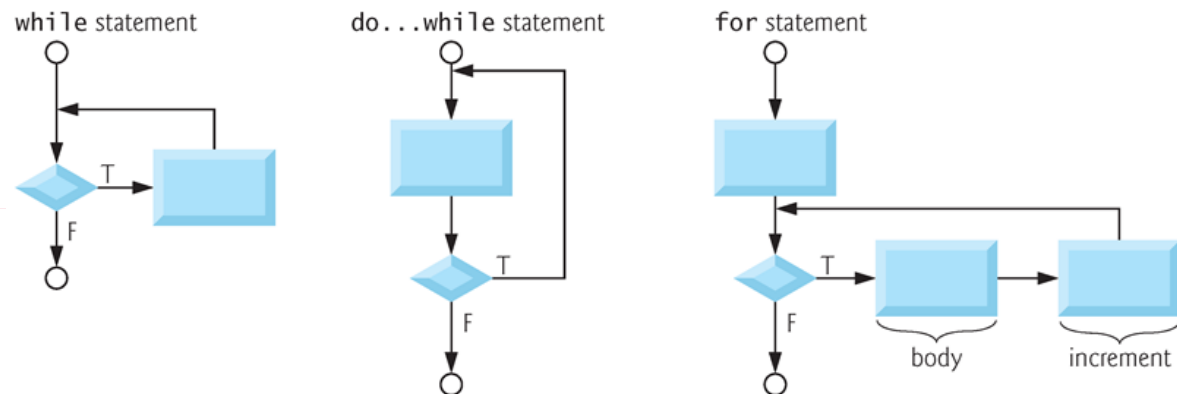


Selection



- Small circles are used in the figure to indicate the *single entry point* and the *single exit point* of each statement.
- Connecting individual flowchart symbols **arbitrarily** can lead to **unstructured** programs.
- The programming profession has chosen to **combine** flowchart symbols to form a **limited** set of control statements, and to build only **structured** programs.
- Combine rules: **stacking** and **nesting**

Repetition



rectangle is action in the flowchart

4.12 Structured Programming Summary (Cont.)

Rules for forming structured programs

1. Begin with the “simplest flowchart” (Fig. 4.19).
2. (“Stacking” rule) Any rectangle (action) can be replaced by *two* rectangles (actions) in sequence.
3. (“Nesting” rule) Any rectangle (action) can be replaced by *any* control statement (sequence, if, if...else, switch, while, do...while or for).
4. Rules 2 and 3 may be applied as often as you like and in *any* order.

Fig. 4.18 | Rules for forming structured programs.

- Applying the rules of Fig. 4.18 always results in a **structured** flowchart with a neat, building-block appearance.
- Repeatedly applying Rule 2 to the simplest flowchart (Fig. 4.19) results in a **structured** flowchart containing many rectangles in sequence (Fig. 4.20).
- Rule 2 generates a **stack** of control statements; so we call Rule 2 the **stacking rule**.

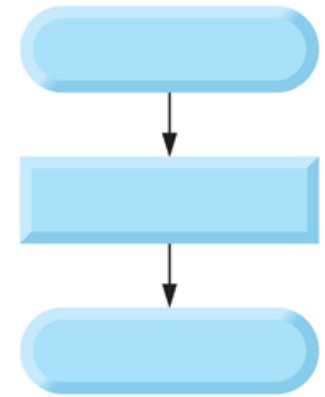


Fig. 4.19 | Simplest flowchart.

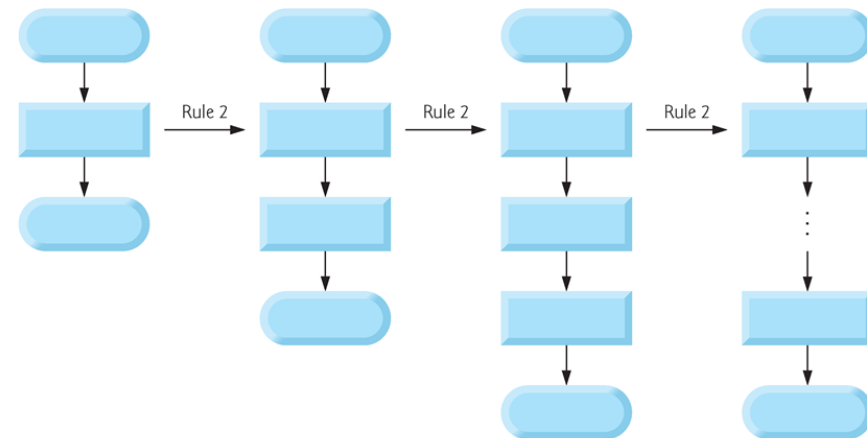


Fig. 4.20 | Repeatedly applying Rule 2 of Fig. 4.18 to the simplest flowchart.

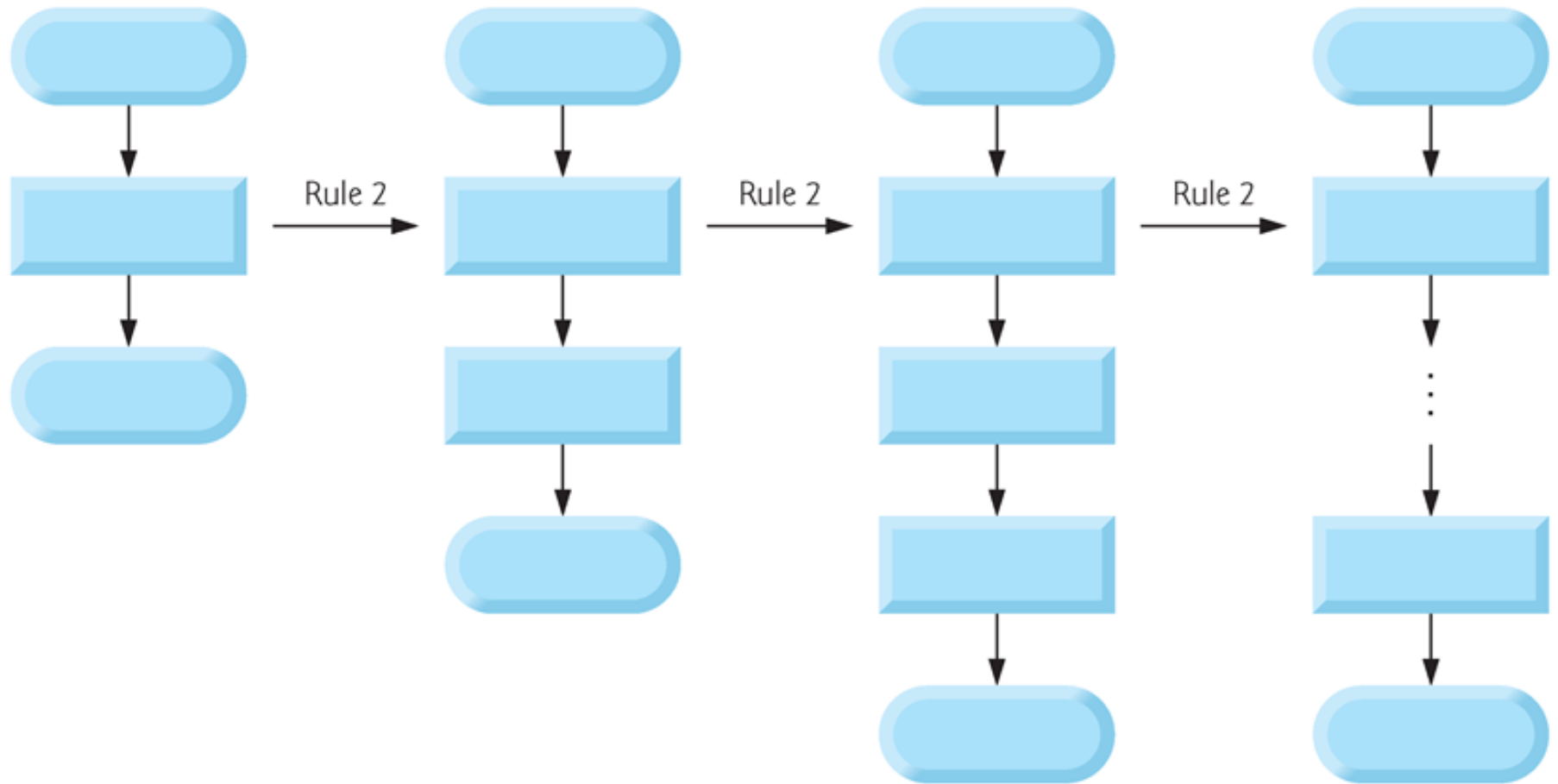


Fig. 4.20 | Repeatedly applying Rule 2 of Fig. 4.18 to the simplest flowchart.

4.12 Structured Programming Summary (Cont.)

3. (“Nesting” rule) Any rectangle (action) can be replaced by *any* control statement (sequence, if, if...else, switch, while, do...while or for).

- Rule 3 is called the **nesting rule**.
- Repeatedly applying Rule 3 to the simplest flowchart results in a flowchart with neatly **nested** control statements.

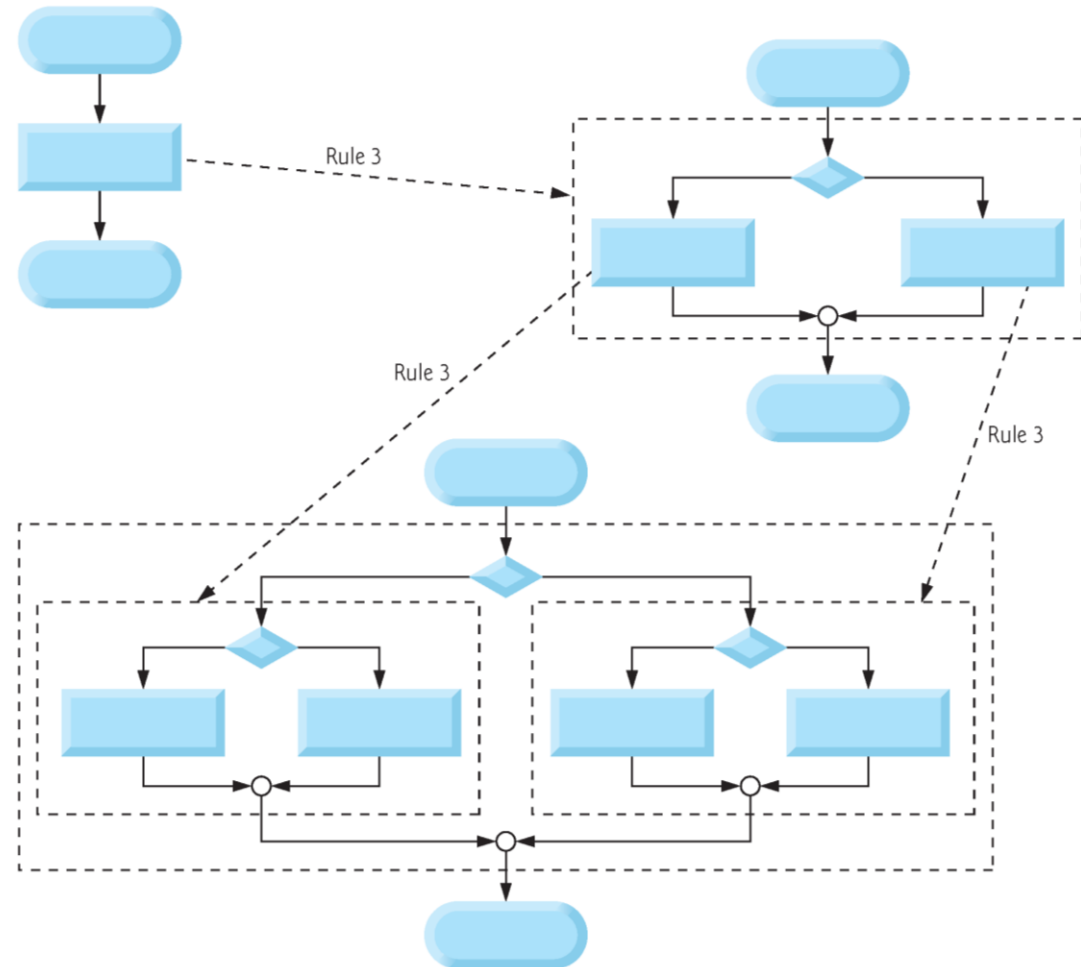


Fig. 4.21 | Applying Rule 3 of Fig. 4.18 to the simplest flowchart.

4.12 Structured Programming Summary (Cont.)

4. Rules 2 and 3 may be applied as often as you like and in *any* order.

- Rule 4 generates **larger**, more involved, and more **deeply** nested structures.

4.12 Structured Programming Summary (Cont.)

- Figure 4.22 shows the kinds of **stacked** building blocks that emerge from applying Rule 2 and the kinds of **nested** building blocks that emerge from applying Rule 3.
- The figure also shows the kind of **overlapped** building blocks that **cannot** appear in structured flowcharts (because of the elimination of the goto statement).

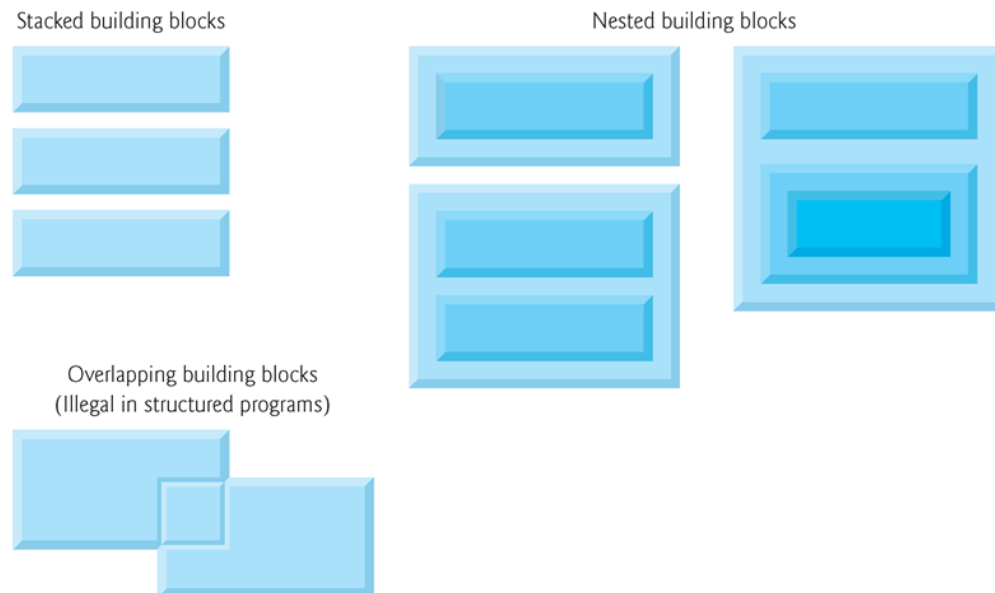


Fig. 4.22 | Stacked, nested and overlapped building blocks.

4.12 Structured Programming Summary (Cont.)

- An **unstructured** flowchart cannot be created.

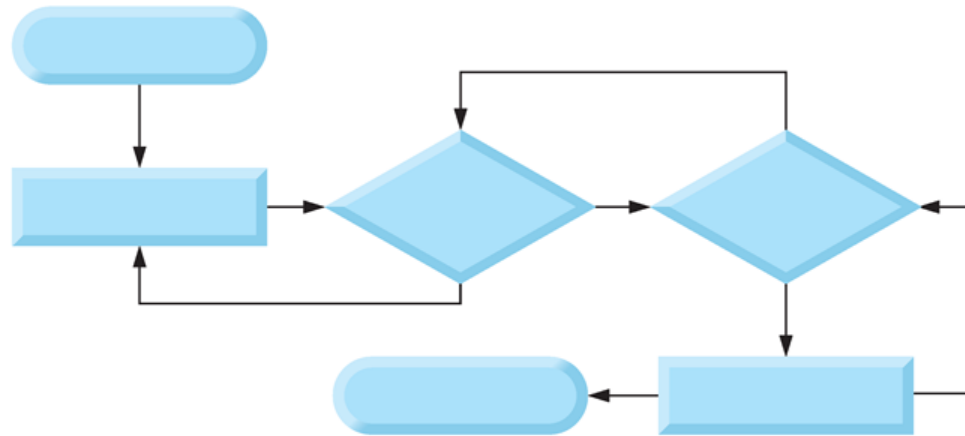


Fig. 4.23 | An unstructured flowchart.

- Structured programming promotes **simplicity**.
- Bohm and Jacopini showed that **only three forms** of control are needed:
 - Sequence
 - Selection
 - Iteration

Program Exercise 1

4.10 (*Conversion Celsius to Fahrenheit*) Write a program that converts temperatures from 30°C to 50°C to the Fahrenheit scale. The program should print a table displaying temperatures in the two scales side by side. [Hint: $^{\circ}\text{F} = \frac{9}{5}\text{C} + 32$]

```
Conversion of temperature from Celsius to Fahrenheit
Celsius Fahrenheit
30      86.00000010.2f
31      87.80000010.2f
32      89.60000010.2f
33      91.40000010.2f
34      93.20000010.2f
35      95.00000010.2f
36      96.80000010.2f
37      98.60000010.2f
38      100.40000010.2f
39      102.20000010.2f
40      104.00000010.2f
41      105.80000010.2f
42      107.60000010.2f
43      109.40000010.2f
44      111.20000010.2f
45      113.00000010.2f
46      114.80000010.2f
47      116.60000010.2f
48      118.40000010.2f
49      120.20000010.2f
50      122.00000010.2f
```


Program Exercise 2

4.26 (*Calculating the Value of π*) Calculate the value of π from the infinite series

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots$$

Print a table that shows the value of π approximated by one term of this series, by two terms, by three terms, and so on. How many terms of this series do you have to use before you first get 3.14? 3.141? 3.1415? 3.14159?

- 最多執行400,000 terms
- $\text{Pi} - 3.14 < 0.000001$
- $\text{Pi} == 3.14$?

376826	3.141590
376827	3.141595
376828	3.141590
376829	3.141595
376830	3.141590
376831	3.141595
376832	3.141590
376833	3.141595
376834	3.141590
376835	3.141595
376836	3.141590
376837	3.141595
376838	3.141590
376839	3.141595
376840	3.141590
376841	3.141595
376842	3.141590
376843	3.141595
376844	3.141590
376845	3.141595
376846	3.141590
376847	3.141595
376848	3.141590
376849	3.141595
376850	3.141590
The terms of this series before first get 3.14 is 1686	
The terms of this series before first get 3.141 is 2028	
The terms of this series before first get 3.1415 is 12098	
The terms of this series before first get 3.14159 is 376850	